College Connect Documentation

Group 1

Jonathan Feurtado, Julian Franzen, Afif Mujibur-Rahman, Thomas Benfield, Zachary Tissue

How to Run

Prerequisite Set-up / Add-on Installations

- Alembic == 1.13.1
- Blinker == 1.7.0
- Click == 8.1.7
- Colorama == 0.4.6
- DateTime == 5.5
- Flask == 3.0.2
- Flask-Login == 0.6.3
- Flask-Mail == 0.9.1
- Flask-Migrate == 4.0.7
- Flask-SQLAlchemy == 3.1.1
- Flask-WTF == 1.2.1
- Greenlet == 3.0.3
- Itsdangerous == 2.1.2
- Jinja2 == 3.1.3
- Mako == 1.3.2
- MarkupSafe == 2.1.5
- SQLAlchemy == 2.0.29
- Typing_extensions == 4.10.0
- Werkzeug == 3.0.1
- WTForms == 3.1.2

Running / Hosting the Website

- "**flask db-init**" is necessary to properly load the forums on a first run.
- The website can be launched with "**flask run**"
    - If the website does not run, it could be a problem with the database being out of date
        - If so run: "flask db migrate" and "flask db upgrade" in that order
    - If problems persist, it may be an issue with the installed add-ons, being that they are either missing or in an out of date version on your end.

    - One of our partners had great difficulty running the site on a MAC in the last week of the project, and we were unable to figure out why. Be wary of running on a MAC or other Apple Device.

Documentation

General Functionality (app.py)

- This application uses python flask which serves to simplify HTML/CSS customization, as well as database and model functionality.
- Flask SQLAlchemy provides the foundation for the database and the models contained within.
    - Each model has attributes which are laid out as columns in tables which are shown as a representation at the bottom of the model code.
    - Flask Migration allows database initialization and new columns to be created
- Flask Forms allow for simplified forms and better interaction between models such as in the case of the forum system.

Sign-up / Log-in

- The signup and login functionality uses flask login in order to easily verify users.
- When a user signs up, the username and password are taken, verified, and added to the database.

User Accounts

- The user accounts system is a database with each row representing a user account
- Each user account has effectively two types of fields , account identification fields and personality fields. The former are set on account creation and cannot be changed, the latter are personality and attribute data a user may add about themselves from the profile page if they so choose. All Personality/Attribute fields are integers for easier and less expensive comparisons within the roommate finder application.
- Identification fields include
    - **Id**: Index of the user account in the database
    - **Username**: The account's display name.
    - **Password**: The account's password, used for logging in.
    - **E-Mail**: The account associated email address, used for logging in and contacting the user via roommate finder
    - A Birthday and Phone Number fields were planned, but were ultimately scrapped.
- Personality / Attribute Fields:
    - **Note: 999** is the universal value for "N/A / Don't Care / etc." and it is the default value of every field listed on account creation.
    - **Gender**: User's Gender
        - 0 - Male
        - 1 - Female

- **Grade**: User's College Grade-level
  - 0 - Freshman
  - 1 - Sophomore
  - 2 - Junior
  - 3 - Senior
- **Religion**: User's Religion
  - 0 - Agnosticism
  - 1 - Atheism
  - 2 - Christianity
  - 3 - Judaism
  - 4 - Islam
  - 5 - Buddhism
  - 6 - Hinduism
- **Drugs**: Whether or not the user is okay with the use of recreational drugs
  - 0 - Yes
  - 1 - No
- Neatness: How Neat the user considers themselves to be
  - 1 - Not neat at all / Cares little about neatness
  - 5 - Very Neat / Cares a lot about Neatness
- **Chronotype**: What Chronotype does the user consider themselves to be
  - 0 - Early Bird
  - 1 - Night Owl
- **About Me**: A text string the user may write on their profile to describe themselves.

Profile Page

- A page where a user's personality / attribute values from the database are displayed.
  - If no data exists or the value is "N/A or Do not Care", it will either appear as a "N/A" , "None", etc.
- An account's personality / attribute fields can be edited here via the "Edit Profile" button
  - The ProfileEdit page contains a form with dropdown menus for the integer attributes and a text box for the About Me field.
  - When the form is submitted, the "/edit" app route and the accountedit() function are called, where in the account's values in the database will be updated to reflect the input values and then the user will be redirected to the profile page where they can see their changes.
    - If the About Me input is left untouched or the value is null, it will ignore the input and keep the About Me field at its original value.

Roommate Finder

- The Roommate finder feature allows the user to input a set of criteria and get a list of accounts from the user database that match the input criteria.

- The user first fills in a form of the personality/attribute values you desire in a roommate (organized into similar dropdown lists to the profile edit page), and once this form is submitted, it will become the “criteria”.
    - The criteria is effectively a dummy account object containing the same personality fields as an account, without identifying information and its data does not persist outside of the Roommate finder feature.
    - **“Don’t Care”** in this form is equivalent to 999
- Once the form is submitted, app.py runs “/roommate” and the roommate() method.
    - First the accounts database is transformed into a temporary list object
    - Then a Rudimentary filter Algorithm is run comparing the criteria’s fields to an account’s field.
        - The algorithm works by having a boolean flag set at the start of a loop.
        - The account’s value will be compared to the user’s criteria. if the user passes, they proceed to the next check. if they fail the check, they are considered an undesirable account and their boolean flag will be set to false and they will skip all following checks and proceed to the next account in the loop.
        - If the criteria’s field is equal to 999 it will “skip” that check, as it is not a personality trait or attribute that matters to the user.
        - If the boolean flag is true at the end of the loop, the user will be added to the roommateList return list.
    - Once the loop is done running, the function returns a list of roommates and reloads the roommate finder page, displaying the users who match your criteria in a list at the bottom of the screen, along with an email to contact them with.
- **Note**: The original roommate finder code (labeled as a mock-up) is present within the project’s github repo, but this is an isolated file that was mainly used for testing and has nothing to do with the project’s functionality.

Forum

- The forum system is broken down into three major parts: the topics, threads in those topics, and posts in the threads.
- The topics are predetermined themes for discussion, they can not be modified by any user and all threads are posted under them.
    - Topics are initialized through a custom command in the terminal
- Threads are the more specific discussions created by individual users under which other users can post responses.
    - When the thread is posted by an authenticated user it displays the username and title for other authenticated users to see.
    - Both threads and posts under those threads are related within the model in order to maintain consistency
    - When a thread is created the authenticated user ID and topic ID are checked in order to make sure that everything is correct.
- Posts form the foundation of discussion in threads.
    - Username and time of posting are provided when a post is made

    - Due to the relationship between models when a thread is deleted by the user who created it all posts in it are deleted.
- Flask login is used to hide thread/post creation functionality from unauthenticated users.